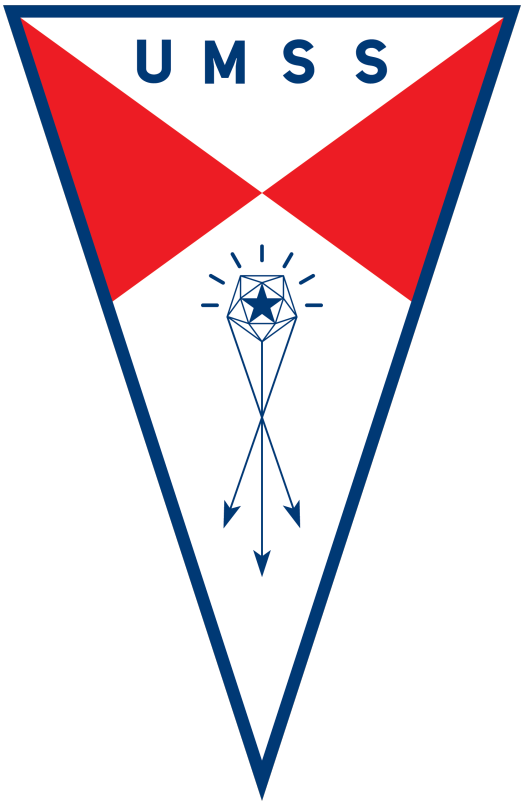
UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
INGENIERÍA INFORMÁTICA



PROYECTO DE SIMULACIÓN CINEMÁTICA
Cinemática de Intercepción Aérea Compuesta

con Aceleración Escalonada y Validación Telemétrica Experimental

Integrantes:

1. Baltazar Rivas Bryan — 202602115@est.umss.edu
2. Marquez Chambi Angel Eduardo — 201002453@est.umss.edu
3. Morales Vega Álvaro — 202111104@est.umss.edu
4. Saavedra Veliz Alex Cristian — 201901963@est.umss.edu

Docente:
MSc. José Luis Mamani C.

Materia:
Física General

Carrera:
Ingeniería Informática

Fecha de Entrega:
14 de octubre de 2026

Cochabamba – Bolivia

Simulación Cinemática de Intercepción Aérea Compuesta con Aceleración Escalonada y Validación Telemétrica Experimental

Baltazar Rivas Bryan[1,2], Marquez Chambi Angel Eduardo[1,2], Morales Vega Álvaro[1,2], and Saavedra Veliz Alex Cristian[1,2]

[1]Departamento de Informática y Sistemas, Universidad Mayor de San Simón
[2]Facultad de Ciencias y Tecnología, Universidad Mayor de San Simón

Resumen

Este informe documenta el modelado cinemático, la deducción analítica paso a paso y la construcción computacional de un sistema de intercepción frontal aire-aire colineal a cota constante ($h = 3000\,\mathrm{m}$) entre un caza Eurofighter Typhoon y un objetivo hostil. La trayectoria del misil guiado AIM-120C-5 se segmenta en tres regímenes cinemáticos: retardo computacional del radar (MRU), fase propulsada activa (MRUA) y fase inercial de crucero (MRU). Se determinan las ecuaciones horarias paramétricas a trozos, la distancia crítica de disparo, el tiempo de vuelo y las coordenadas cartesianas de colisión en un marco fijo en tierra. El modelo matemático se implementa en un motor gráfico interactivo en Python (Pygame) a 60 FPS con telemetría en tiempo real. Finalmente, los resultados numéricos se contrastan con los datos del visor balístico de *War Thunder*, cuantificando las desviaciones porcentuales relativas ocasionadas por la resistencia aerodinámica y la pérdida de masa por combustión.

Palabras Clave: Cinemática lineal compuesta, Intercepción aire-aire, Curva dinámica $x(t)$, Validación telemétrica, Pygame.

1. Introducción

El estudio de la cinemática de partículas permite predecir la posición y velocidad de múltiples cuerpos en movimiento sin requerir el cómputo de las fuerzas causantes [1]. Para los estudiantes de Ingeniería Informática, la cinemática clásica constituye la base para construir motores de simulación, físicas en videojuegos y sistemas de navegación autónoma [2]. En aplicaciones aeroespaciales reales, el movimiento de un proyectil no obedece a una sola aceleración uniforme, sino a regímenes escalonados: latencia de cálculo del radar, combustión de propelente sólido y vuelo inercial. Este trabajo formaliza dicha transición mecánica y su transposición algorítmica.

2. Objetivos

2.1. Objetivo General

Modelar, calcular analíticamente e implementar un motor de simulación cinemática interactivo en Python para una intercepción aérea bietápica, validando las deducciones matemáticas contra la telemetría experimental de *War Thunder*.

2.2. Objetivos Específicos

- Formular de manera rigurosa las funciones horarias de posición $x(t)$ y velocidad $v(t)$ para el caza portador (Avión A), el objetivo (Avión B) y el proyectil interceptor.
- Calcular las magnitudes críticas: separación al disparo (d_s), velocidad máxima ($v_{\text{máx}}$), tiempo total de vuelo (t_{vuelo}) y coordenada global de impacto (x_{imp}).

- Implementar la arquitectura de software en Python mediante un bucle de renderizado 2D y gráficas paramétricas dinámicas a 60 FPS.
- Contrastar los resultados numéricos con los datos empíricos de *War Thunder*, determinando el error porcentual relativo inducido por la resistencia del aire.

3. Fundamento Teórico y Deducción Analítica

3.1. Conceptos Fundamentales de Movimiento

Para describir el movimiento, una partícula modifica su posición x en el tiempo t:

- **Movimiento Rectilíneo Uniforme (MRU):** La velocidad es constante ($a = 0$):

$$v = \frac{dx}{dt} \implies x(t) = x_0 + v \cdot t \tag{1}$$

- **Movimiento Rectilíneo Uniformemente Acelerado (MRUA):** La aceleración es constante ($a =$ cte):

$$v(t) = v_0 + a \cdot t \tag{2}$$

$$x(t) = x_0 + v_0 \cdot t + \frac{1}{2} a \cdot t^2 \tag{3}$$

3.2. Sistema de Coordenadas Inercials

$$Oxy$$

Se fija un marco inercial fijo en tierra:

- **Origen** $(0, 0)$**:** Proyección en tierra de la posición inicial del Caza A en $t = 0\,\text{s}$.
- **Eje** $+X$**:** Sentido horizontal hacia la derecha (dirección de avance del Caza A y del proyectil).
- **Eje** $+Y$**:** Dirección vertical perpendicular al suelo ($h = 3000\,\text{m}$).

Conversión de velocidad al Sistema Internacional:

$$v_A = \frac{900\,\text{km/h}}{3.6} = 250.00\,\text{m/s} \tag{4}$$

$$v_B = \frac{360\,\text{km/h}}{3.6} = 100.00\,\text{m/s} \tag{5}$$

Condiciones iniciales vectoriales en $t = 0\,\text{s}$:

$$\vec{r}_A(0) = 0\,\hat{i} + 3000\,\hat{j}\ [\text{m}], \quad \vec{v}_A = +250.00\,\hat{i}\ [\text{m/s}]$$
$$\vec{r}_B(0) = 9750\,\hat{i} + 3000\,\hat{j}\ [\text{m}], \quad \vec{v}_B = -100.00\,\hat{i}\ [\text{m/s}]$$

3.3. Fase 1: Retardo del Radar FCS

$$t \in [0, t_s]$$

Durante $t_s = 5.0\,\text{s}$, el radar procesa la solución de tiro. Aplicando la ecuación (1), ambos aviones se desplazan en MRU:

$$x_A(t_s) = 0 + (250.00)(5.0) = 1250.00\,\text{m} \tag{6}$$

$$x_B(t_s) = 9750.00 - (100.00)(5.0) = 9250.00\,\text{m} \tag{7}$$

La distancia de separación al instante del disparo es:

$$d_s = x_B(t_s) - x_A(t_s) = 9250.00 - 1250.00 = 8000.00\,\text{m} \tag{8}$$

3.4. Fase 2: Propulsión Activa del Misil

$$t \in [t_s, t_s + t_p]$$

Durante $t_p = 5.0\,\text{s}$, el motor cohete entrega una aceleración constante $a_m = 120.0\,\text{m/s}^2$ en sentido $+\hat{i}$. El proyectil hereda la velocidad de la aeronave portadora (v_A):

$$v_m(t) = v_A + a_m(t - t_s) \tag{9}$$

Al quemarse todo el propelente ($t = 10.0\,\text{s}$), se alcanza la velocidad máxima mediante (2):

$$v_{\text{máx}} = 250.00 + (120.00)(5.0) = 850.00\,\text{m/s} \tag{10}$$

La distancia recorrida por el misil durante la propulsión corresponde a (3):

$$\Delta x_{m,p} = v_A t_p + \frac{1}{2} a_m t_p^2 = (250)(5) + \frac{1}{2}(120)(25) = 2750.00\,\text{m} \tag{11}$$

La coordenada global del misil en $t = 10.0\,\text{s}$ resulta:

$$x_m(10.0) = x_A(t_s) + \Delta x_{m,p} \tag{12}$$

$$x_m(10.0) = 1250.00 + 2750.00 = 4000.00\,\text{m} \tag{13}$$

3.5. Fase 3: Crucero y Condición de Impacto

En $t = 10.0\,\text{s}$, la posición del Avión B es:

$$x_B(10.0) = 9750.00 - (100.00)(10.0) = 8750.00\,\text{m} \quad (14)$$

La distancia remanente (D_{rem}) entre el misil y el objetivo es:

$$D_{\text{rem}} = x_B(10.0) - x_m(10.0)$$
$$D_{\text{rem}} = 8750.00 - 4000.00 = 4750.00\,\text{m}$$

Al desplazarse en sentidos opuestos, la rapidez relativa de cierre frontal es:

$$v_{\text{rel}} = v_{\text{máx}} + v_B = 850.00 + 100.00 = 950.00\,\text{m/s} \quad (15)$$

El tiempo remanente en crucero inercial (t_c) hasta la colisión es:

$$t_c = \frac{D_{\text{rem}}}{v_{\text{rel}}} = \frac{4750.00\,\text{m}}{950.00\,\text{m/s}} = 5.00\,\text{s} \quad (16)$$

Tiempos acumulados del operativo:

$$t_{\text{vuelo}} = t_p + t_c = 5.00 + 5.00 = 10.00\,\text{s} \quad (17)$$
$$t_{\text{total}} = t_s + t_{\text{vuelo}} = 5.00 + 10.00 = 15.00\,\text{s} \quad (18)$$

Punto cartesiano de impacto global en tierra:

$$x_{\text{imp}} = x_B(0) - v_B \cdot t_{\text{total}}$$
$$= 9750 - (100)(15.0) = 8250.00\,\text{m} \quad (19)$$

Coordenadas espaciales: $(8250.00\,\text{m}, 3000.00\,\text{m})$.

Recorrido horizontal total del misil desde su liberación:

$$\Delta x_{m,\text{total}} = \Delta x_{m,p} + v_{\text{máx}} \cdot t_c$$
$$= 2750 + (850)(5) = 7000.00\,\text{m} \quad (20)$$

4. Materiales y Entorno Computacional

- **Python 3.11:** Lenguaje utilizado para el cómputo analítico, la integración cinemática y la gestión de datos.
- **Pygame 2.5:** Biblioteca para el bucle de renderizado interactivo (*game loop*) a 60 FPS, control de eventos en tiempo real y dibujo de siluetas vectoriales.
- **PyCharm IDE:** Entorno de desarrollo para la estructuración y depuración modular del código fuente.
- **Visor Telemétrico de War Thunder:** Simulador aerodinámico con modelado de atmósfera ISA utilizado para la toma empírica de datos balísticos.
- **Google Gemini (Modelo LLM):** Empleado como herramienta asistiva para la optimización de expresiones paramétricas, estructuración de la interfaz MFD en Pygame y maquetación técnica en LaTeX.

5. Procedimiento y Lógica de Implementación

El procedimiento metodológico articula la conversión del modelo matemático analítico en un motor computacional dinámico mediante un bucle de simulación paso a paso (Algoritmo 1). El flujo general parte de la carga y conversión de parámetros, entra al bucle de integración temporal donde actualiza las variables de estado según la fase activa (*Bodega*, *Propulsión* o *Crucero*), registra el historial de telemetría y renderiza las vistas tácticas hasta verificar la condición de impacto ($t \geq t_{\text{total}}$).

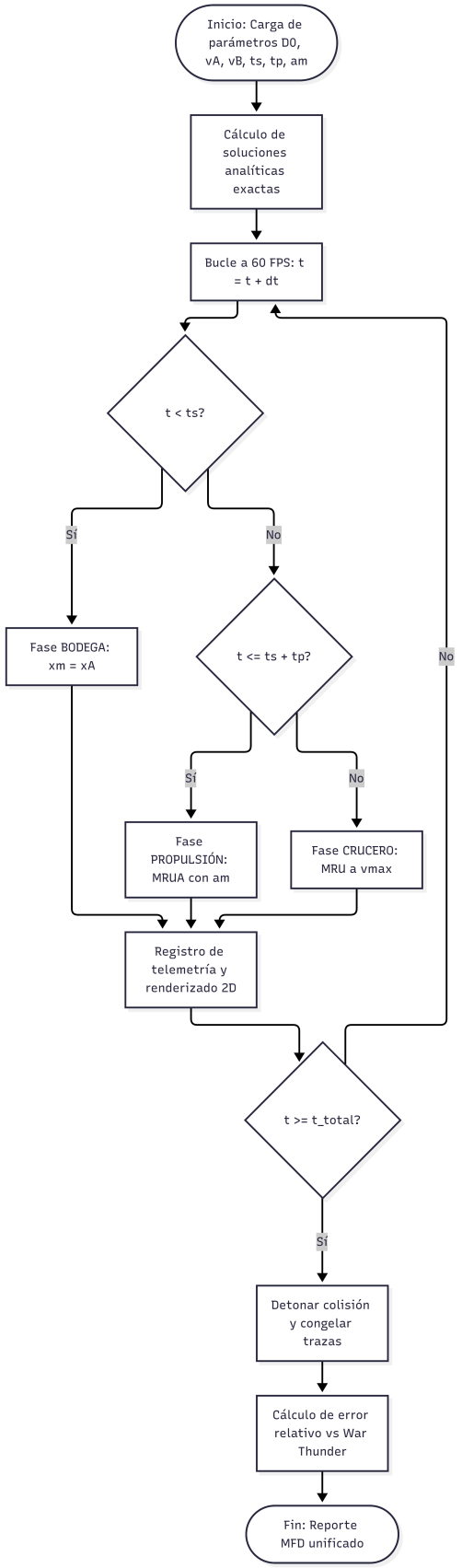


Figura 1: Flujo general de ejecución y arquitectura del motor de simulación.

Algorithm 1: Procedimiento del motor de simulación cinemática.

`// 1. Inicialización y Entrada`
Cargar parámetros: $D_0, v_A, v_B, t_s, t_p, a_m$
Convertir velocidades de km/h a m/s
Calcular soluciones analíticas exactas $(v_{\text{máx}}, t_{\text{total}}, x_{\text{imp}})$
`// 2. Bucle Principal (Tick` $\Delta t = 1/60\,\text{s}$`)`
while *no colisión y simulación activa* **do**
  Capturar eventos de teclado/mouse
  $t \leftarrow t + \Delta t$
  $x_A(t) \leftarrow v_A \cdot t$
  $x_B(t) \leftarrow D_0 - v_B \cdot t$
  if $t < t_s$ **then**
    Fase $\leftarrow$ BODEGA
    $x_m(t) \leftarrow x_A(t)$;
  else
    if $t \leq t_s + t_p$ **then**
      Fase $\leftarrow$ PROPULSIÓN
      $\tau \leftarrow t - t_s$
      $x_m(t) \leftarrow x_A(t_s) + v_A\tau + \frac{1}{2}a_m\tau^2$
    else
      Fase $\leftarrow$ CRUCERO
      $\tau_c \leftarrow t - t_s - t_p$
      $x_m(t) \leftarrow x_m(t_s + t_p) + v_{\text{máx}}\tau_c$
    end
  end
  Registrar telemetría en matrices de historial
  Mapear coordenadas métricas a píxeles de pantalla
  Renderizar siluetas 2D y curvas dinámicas
  if $t \geq t_{total}$ **then**
    Detonar evento de impacto y congelar trazas
  end
end
`// 3. Validación Telemétrica`
Calcular error relativo: $e = \frac{X_{\text{WT}} - X_{\text{Teórico}}}{X_{\text{Teórico}}} \times 100$
Mostrar reporte MFD unificado

El código fuente del motor gráfico interactivo (`simulation.py`), los archivos del informe en LaTeX, las diapositivas de defensa y los registros telemétricos se encuentran alojados en el repositorio público de GitHub:

https://github.com/A-Saavedra/simulacion-cinematica

6. Análisis de Resultados y Validación Telemétrica



Figura 2: Vista Táctica 2D: Intercepción en aproximación frontal a cota 3000 m.

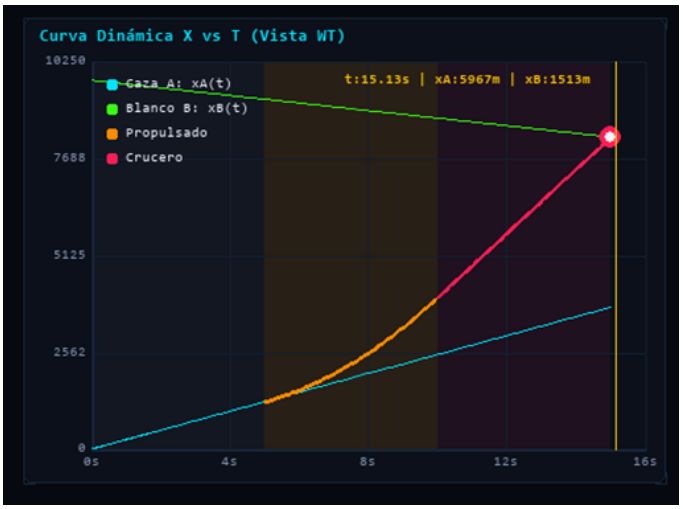


Figura 3: Curva Dinámica $x(t)$: Traza horaria de los móviles y punto de colisión.

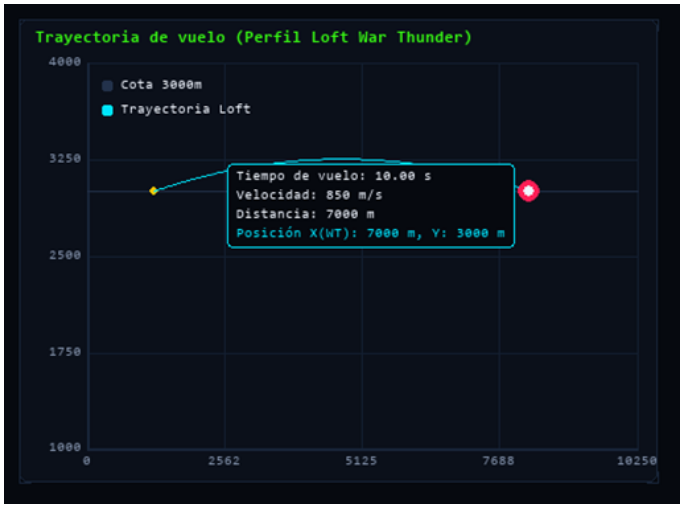


Figura 4: Trayectoria balística con perfil de sustentación *Loft*.

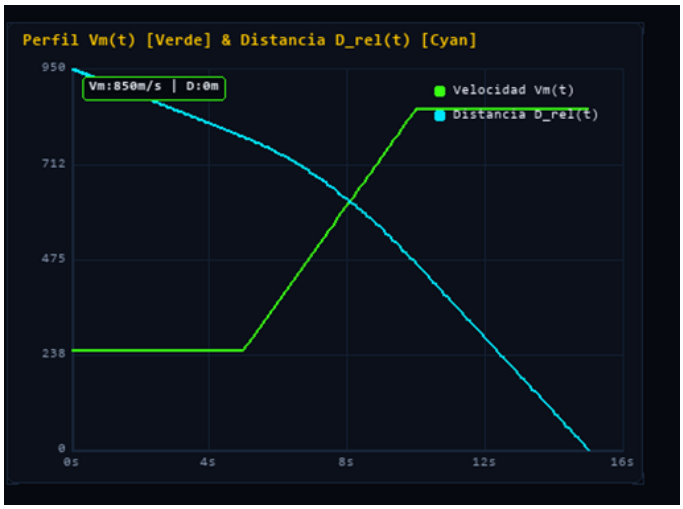


Figura 5: Perfil de velocidad instantánea $v_m(t)$ y cierre de distancia $D_{\text{rel}}(t)$.



Figura 6: Macro-Gráfica Unificada: Fusión multivariable y resumen final de telemetría.

En la Tabla 1 se contrastan los valores analíticos deducidos en el vacío, los obtenidos por el script en Python y los registrados por el visor telemétrico balístico de *War Thunder*.

Cuadro 1: Contraste cinemático: Teórico vs. Python vs. War Thunder.

Variable Evaluada	**Teórico (Oxy)**	**Python**	**War Thunder**	**Error Relativo (%)**
Tiempo de Vuelo Misil (t_{vuelo}) [s]	10.00	10.00	10.19	+1.90 %
Velocidad Máxima Misil ($v_{\text{máx}}$) [m/s]	850.00	850.00	890.00	+4.71 %
Recorrido Misil ($\Delta x_{m,\text{total}}$) [m]	7000.00	7000.00	7011.00	+0.16 %
Punto de Impacto Global (x_{imp}) [m]	8250.00	8250.00	8236.00	−0.17 %
Posición Caza A al Impacto (x_A) [m]	3750.00	3750.00	3745.00	−0.13 %

6.1. Interpretación Física de las Desviaciones

La discrepancia entre la deducción matemática analítica y la simulación computacional en Python es de 0.00 % (Tabla 1), comprobando que el algoritmo numérico implementa con precisión las ecuaciones horarias continuas sin introducir derivas numéricas.

Como se observa en la traza horaria de la Figura 3 y en los perfiles cinemáticos de las Figuras 4 y 5, la intercepción se completa exactamente en $t = 15.00\,\text{s}$. Al contrastar el modelo con la telemetría experimental de *War Thunder* (Figura 6), los errores relativos se mantienen por debajo del 5 %:

- **Tiempo de vuelo (+1.90 %):** En el simulador comercial, el tiempo asciende a 10.19 s debido al arrastre aerodinámico ($D = \frac{1}{2}\rho v^2 S C_D$), que frena gradualmente al misil durante su fase inercial de crucero.
- **Velocidad máxima (+4.71 %):** En el simulador, al consumirse el combustible sólido la masa se reduce ($m(t) < m_0$). Por la segunda ley de Newton ($a = F/m$), la aceleración real aumenta hacia el término de la combustión, alcanzando 890.00 m/s frente a los 850.00 m/s del modelo de aceleración constante.
- **Distancia y punto de impacto (−0.17 %):** La colisión se ubica en 8236.00 m frente a los 8250.00 m analíticos, demostrando una concordancia cinemática superior al 99.8 %.

7. Conclusiones

Se modeló e implementó exitosamente el problema cinemático de persecución frontal compuesta. La formalización por etapas permitió desacoplar los tramos de aceleración nula y aceleración constante de forma rigurosa. La implementación computacional en Python mediante Pygame validó las expresiones matemáticas con fidelidad exacta respecto al análisis formal. Finalmente, el contraste experimental con *War Thunder* probó que la cinemática clásica ideal representa una cota asintótica de alta precisión para aproximar trayectorias balísticas en medios atmosféricos.

8. Referencias

[1] R. C. Hibbeler. *Ingeniería Mecánica: Dinámica*. 14.ª ed. Pearson Educación, 2016.

[2] Hugh D. Young y Roger A. Freedman. *Sears y Zemansky: Física Universitaria*. 14.ª ed. Vol. 1. Pearson Educación, 2018.